CYBERSAFE

Cybersecurity Awareness Chatbox Using Python

Project Report

Submitted by: Prabhjot Kaur

Course / Class: BCA

College: Guru Nanak Dev University

Submission date: 23 October,2026

1. Abstract

CyberSafe is a desktop chatbot developed in Python to promote cybersecurity awareness. Users can ask simple questions about common online risks and receive prepared educational answers. The project uses Tkinter to provide a graphical interface and a keyword-based knowledge base to select responses.

2. Introduction

Cybersecurity protects computers, networks, applications, and information from unauthorised access, misuse, or damage. Many incidents begin with human mistakes, such as clicking suspicious links or reusing passwords. An accessible chatbot can help students learn basic protective practices.

3. Problem Statement

New users may not know how to recognise phishing, protect passwords, or respond to a compromised account. CyberSafe offers a simple, interactive place to learn introductory safety guidance.

4. Objectives

• Provide basic cybersecurity information through a chat interface.
• Explain phishing, malware, passwords, Wi-Fi safety, backups, and updates.
• Demonstrate Python dictionaries, functions, conditions, loops, and GUI programming.
• Build a project that can be extended with additional topics.

5. Scope

The project is an educational prototype for students and general users. It responds to selected keywords and provides general guidance. It does not conduct penetration testing, scan a network, or verify a website in real time.

6. Tools and Technologies

Programming language: Python
GUI library: Tkinter
Knowledge storage: Python dictionary
Message timestamps: datetime module
External dependencies: None; Tkinter must be available in the Python installation.

7. System Design

The user types a message in the GUI. The program converts the message to lowercase, checks it against supported keywords, selects the matching answer, and displays the response. If no keyword matches, the chatbot suggests using the help command.

8. Main Modules

GUI module: Displays the chat window, message history, input field, and Send button.
Knowledge-base module: Stores cybersecurity topics and prepared answers.
Response module: Matches user input with keywords and returns a response.
Message module: Adds speaker labels and timestamps.

9. Implementation

The program is saved as *chatbox.py*. Tkinter creates the desktop interface. A dictionary named KNOWLEDGE_BASE stores responses. The get_reply() function finds a matching keyword and returns the relevant text. The interface accepts messages using the Send button or Enter key.

10. Installation and Execution

1. Install Python 3.9 or newer.
2. Confirm Tkinter is available (it is included with many Python installations).
3. Save the code as chatbox.py.
4. Open a terminal in the project folder.
5. Run *python chatbox.py*.
6. Ask a question or type *help* to view supported topics.

11. Testing

Test the chatbot with known inputs such as “What is phishing?”, “How can I make a strong password?”, “Explain malware”, and “help”. Confirm that each produces a relevant answer. Test an unsupported question to verify that the fallback message appears. Test an empty input to ensure no blank message is added.

12. Limitations

The chatbot uses keyword matching rather than artificial intelligence. It may not understand spelling mistakes, complex questions, or synonyms that are not in its knowledge base. Its guidance is general and should not be treated as a complete security assessment or professional incident-response service.

13. Future Enhancements

Possible improvements include a larger editable knowledge base, a quiz mode, chat-history export, multilingual responses, a web interface using Flask, and integration with a reputable threat-intelligence API. Any URL-checking feature should clearly explain its data sources and privacy implications.

14. Responsible Use and Safety

Do not enter real passwords, OTPs, or private information into a demonstration chatbot. Use security tools only on systems you own or have explicit permission to test. For real incidents, contact the relevant service through official support channels. In India, use the official cybercrime reporting portal and verify current official reporting instructions.

15. Conclusion

CyberSafe demonstrates how Python can be used to create a simple cybersecurity awareness assistant. It provides a beginner-friendly GUI and reusable responses about common threats. The project can be expanded as the student's Python and cybersecurity knowledge grows.

Appendix A — Example Questions

What is phishing? • Explain malware • Is public Wi-Fi safe? • How do I protect my account? • help